

实验 3：基于 Shamir 秘密共享的安全多方平均值计算

数据安全第三次实验报告

姓名：郭佳成 学号：2311990 专业：密码科学与技术

代码：<https://github.com/Fighting05/DataSecurity>

2026 年 5 月 2 日

1 实验要求

参考教材实验 4.3.3（投票统计实例），实现三个人对他们拥有的数据的平均值的计算。

场景描述：假设有三个同学 Student1、Student2 和 Student3 各持有一个私有整数数据，希望在不向对方（以及计算方）泄露各自数据的前提下，共同计算这三个数据的平均值。

2 实验环境

- 操作系统：Windows 11
- 开发语言：Python 3.9
- 依赖库：无需额外安装，使用 Python 标准库

3 实验原理

3.1 安全多方计算概述

安全多方计算（Secure Multi-Party Computation, MPC）是密码学的一个重要分支，允许多个参与方在不泄露各自私有输入的前提下，联合计算一个函数的输出。其通用框架如图 1 所示：数据方各持有私有数据，通过交互协议将数据安全地传递给计算方，

由计算方完成计算后仅将结果返回给结果方，整个过程中任何一方都无法得知其他方的原始数据。

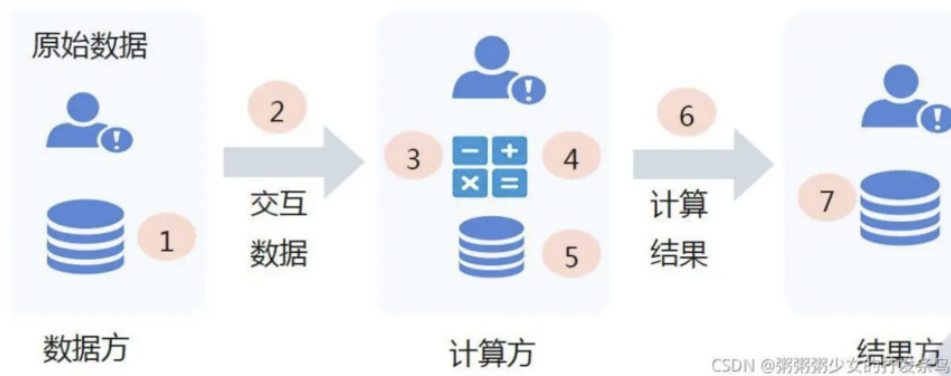


图 1: 安全多方计算通用框架：数据方通过交互将数据传给计算方，结果方得到计算结果

3.2 Shamir 门限秘密共享

Shamir 于 1979 年基于多项式插值算法设计了 Shamir(t, n) 门限秘密共享体制。对于要共享的秘密 s ，其核心步骤如下：

1. **多项式构造**：选择 $t - 1$ 次随机多项式 $f(x) = s + a_1x + a_2x^2 + \cdots + a_{t-1}x^{t-1} \pmod{p}$ ，其中常数项即为秘密 s ，其余系数 $a_1, \dots, a_{t-1}$ 随机选取。
2. **份额分发**：选取 n 个不同的 x 值（通常为 $1, 2, \dots, n$ ），计算对应的 $f(x_i)$ 作为秘密份额，将 $(x_i, f(x_i))$ 发送给第 i 个参与者。
3. **秘密重构**：使用任意 t 个份额，通过拉格朗日插值恢复 $f(0) = s$ 。

本实验采用 (2,3) 门限，即 $t = 2, n = 3$ ，多项式为一次多项式 $f(x) = s + r \cdot x$ ，只需任意两个份额即可重构秘密。

拉格朗日插值公式：已知 t 个点 $(x_0, y_0), \dots, (x_{t-1}, y_{t-1})$ ，恢复 $f(0)$ 为：

$$f(0) = \sum_{i=0}^{t-1} y_i \prod_{j \neq i} \frac{-x_j}{x_i - x_j} \pmod{p}$$

其中除法在模 p 意义下为乘以乘法逆元，利用费马小定理计算： $a^{-1} \equiv a^{p-2} \pmod{p}$ 。

3.3 安全多方平均值计算协议

本实验的协议直接参照教材 4.3.3 投票统计实例，将“统计投票总和”推广为“计算数据平均值”，协议示意图如图 2 所示。

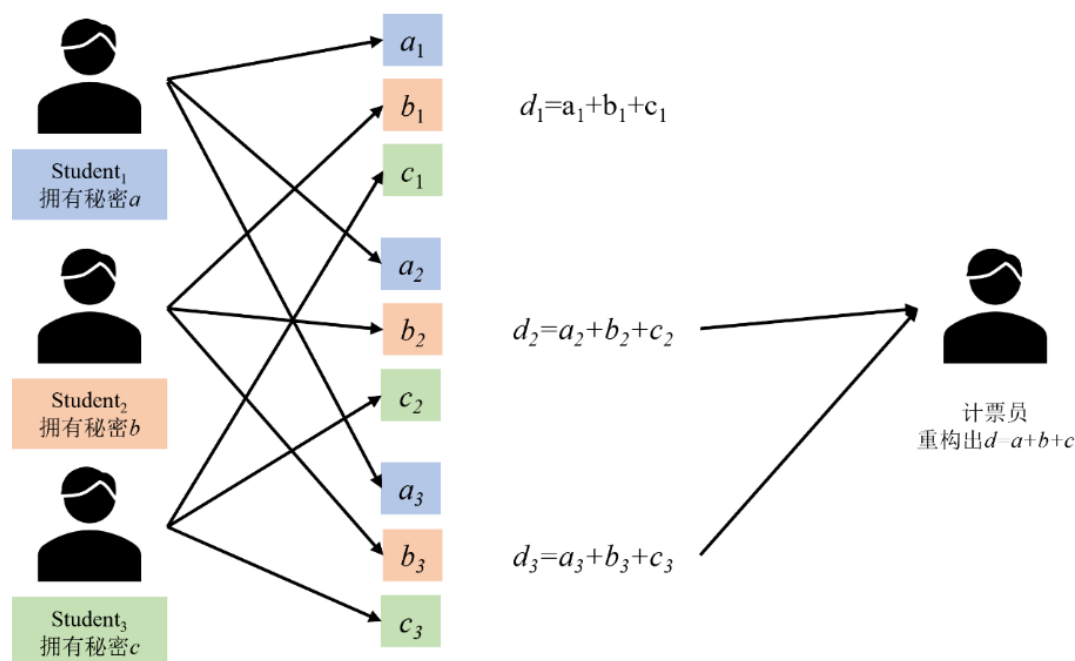


图 2: 协议流程: 三个学生各将私有数据用 (2,3) 门限 Shamir 分享, 计票员重构总和后求平均值

协议共分四个步骤:

步骤 1 一秘密分发: 每个学生 i ($i = 1, 2, 3$) 将自己的私有数据 s_i 用 (2,3) Shamir 秘密共享分享给全部三方 (含自己)。具体地, 构造随机一次多项式 $f_i(x) = s_i + r_i \cdot x \pmod{p}$, 计算三份份额:

$$(j, f_i(j)), \quad j = 1, 2, 3$$

将 $f_i(j)$ 发送给 Student _{j} , 记为 $student_i_j.txt$ 。

步骤 2 一本地聚合: 每个学生 j 收集所有人 (含自己) 给自己的份额, 相加得到聚合值:

$$d_j = f_1(j) + f_2(j) + f_3(j) \pmod{p}$$

由于多项式加法的同态性, d_j 正是总和多项式 $F(x) = f_1(x) + f_2(x) + f_3(x)$ 在点 j 处的值, 而 $F(0) = s_1 + s_2 + s_3$ 。

步骤 3 一重构总和: 计票员选取任意两个学生 (本实验取学生 2 和学生 3) 的聚合值 d_2, d_3 , 用拉格朗日插值恢复 $F(0) = s_1 + s_2 + s_3$ 。

步骤 4 一求平均值:

$$\text{avg} = (s_1 + s_2 + s_3) \times 3^{-1} \pmod{p}$$

其中 $3^{-1} \pmod{p}$ 为 3 的乘法逆元, 由费马小定理计算。

与投票统计的异同：

- 相同之处：协议结构完全一致——秘密分发、本地聚合、拉格朗日重构，代码框架直接复用。
- 不同之处：投票统计恢复总票数即为最终结果；平均值计算额外增加一步，将总和乘以参与方数量的乘法逆元。

3.4 本实现与教材的对应关系

教材（投票统计 4.3.3）	本实现（平均值计算）	说明
ss_student.py 输入投票值	ss_student.py 输入私有数据	接口相同，语义扩展
count_student.py 本地聚合	compute_local.py 本地聚合	逻辑完全一致
vote_counter.py 重构票数	average_counter.py 重构并求均值	增加除以 3 的步骤
无	run_test.py 自动化测试	新增，方便验证

4 代码实现

4.1 代码结构

本次实验代码共分为 5 个文件：

- ss_function.py: 通用函数库（快速幂、多项式构造、多项式求值、拉格朗日重构）
- ss_student.py: 学生端—秘密份额分发，将私有数据拆分为 3 份发给另外两方
- compute_local.py: 学生端—本地聚合，汇总收到的所有份额
- average_counter.py: 计票端—读取任意两个聚合值，重构总和并计算平均值
- run_test.py: 自动化测试脚本，模拟完整协议流程

4.2 核心函数（ss_function.py）

```
1 def quickpower(a, b, p):
2     """快速幂：计算  $a^b \bmod p$ """
3     a = a % p
4     ans = 1
5     while b != 0:
```

```

6         if b & 1:
7             ans = (ans * a) % p
8             b >>= 1
9             a = (a * a) % p
10        return ans
11
12 def restructure_polynomial(x, fx, t, p):
13     """拉格朗日插值：由 t 个点 (x[i], fx[i]) 恢复 f(0)"""
14     ans = 0
15     for i in range(t):
16         fx[i] = fx[i] % p
17         fxi = 1
18         for j in range(t):
19             if j != i:
20                 # L_i(0) = prod_{j!=i} (-x[j]) / (x[i] - x[j])
21                 fxi = (-1 * fxi * x[j]
22                      * quickpower(x[i] - x[j], p - 2, p)) % p
23         fxi = (fxi * fx[i]) % p
24         ans = (ans + fxi) % p
25     return ans

```

Listing 1: 快速幂与拉格朗日插值重构

4.3 学生端：秘密份额分发 (ss_student.py)

```

1 id = int(input("请输入参与方id (1/2/3): "))
2 s = int(input(f"请输入 student_{id} 的私有数据 s: "))
3
4 # 构造一次多项式 f(x) = s + r*x, t-1=1 个随机系数
5 f = ss_f.get_polynomial(s, 1, p, str(id))
6
7 # 计算三份份额 f(1), f(2), f(3)
8 shares_x = [1, 2, 3]
9 shares_y = [ss_f.count_polynomial(f, x, p) for x in shares_x]
10
11 # 写入文件 student_{id}_{j}.txt, 发送给 Student_j
12 for j in range(1, 4):
13     with open(f"student_{id}_{j}.txt", "w") as fp:
14         fp.write(str(shares_y[j - 1]))

```

Listing 2: 学生端秘密份额分发

4.4 学生端：本地聚合 (compute_local.py)

```
1 id = int(input("请输入参与方id (1/2/3): "))
2
3 # 读取三方各给自己的份额: student_1_id, student_2_id, student_3_id
4 data = []
5 for i in range(1, 4):
6     with open(f"student_{i}_{id}.txt", "r") as f:
7         data.append(int(f.read()))
8
9 # 聚合: d_id = sum(所有份额) mod p, 利用同态加法性质
10 d = sum(data) % p
11
12 with open(f"d_{id}.txt", "w") as f:
13     f.write(str(d))
14 print(f"Student_{id} 聚合值 d_{id} = {d}")
```

Listing 3: 学生端本地聚合

4.5 计票端：平均值计算 (average_counter.py)

```
1 # 读取任意两个学生的聚合值 (取学生2和学生3)
2 x_vals = [2, 3]
3 d_vals = [int(open(f"d_{i}.txt").read()) for i in x_vals]
4
5 # 拉格朗日插值恢复  $F(0) = s_1 + s_2 + s_3$ 
6 total = ss_f.restructure_polynomial(x_vals, d_vals, 2, p)
7 print(f"重构总和: {total}")
8
9 # 平均值 = total *  $3^{-1} \bmod p$ 
10 inv_3 = pow(3, p - 2, p) # 费马小定理求3的乘法逆元
11 avg = (total * inv_3) % p
12 print(f"平均值: {avg}")
```

Listing 4: 计票端平均值计算

5 运行结果

激活 conda 环境后运行自动化测试脚本：

```
1 conda activate new
2 cd d:\大三下\数据安全\Lab3
3 python run_test.py
```

测试数据：Student1 = 10, Student2 = 20, Student3 = 30, 预期平均值为 $(10 + 20 + 30)/3 = 20$ 。

测试脚本按协议顺序依次执行三个步骤，完整输出如图 ?? 所示。

```
D:\大三下\数据安全\Lab3>python run_test.py
=====
基于Shamir秘密共享的安全多方平均值计算
输入：Student1=10, Student2=20, Student3=30
预期平均值：20
=====

【步骤1】秘密份额分发
Student1: f1(x)=10+894853552x  份额=(894853552, 789707107, 684560652)
Student2: f2(x)=20+22346654x  份额=(22346674, 44693328, 67039982)
Student3: f3(x)=30+305187124x  份额=(305187154, 610374278, 915561402)

【步骤2】本地聚合
Student1: d_1 = 894853552+22346674+305187154 mod p = 222387383
Student2: d_2 = 789707107+44693328+610374278 mod p = 444774706
Student3: d_3 = 684560652+67039982+915561402 mod p = 667162029

【步骤3】计票端 - 拉格朗日插值重构
取 d_2=444774706, d_3=667162029
重构总和 F(0) = 60
平均值 = 60 * 3^(-1) mod p = 20

=====
✓ 测试通过！ 平均值 = 20 (预期 20)
=====

D:\大三下\数据安全\Lab3>
```

图 3: run_test.py 完整运行结果：秘密份额分发、本地聚合、平均值重构全流程

步骤 1 中，每个学生生成随机一次多项式 $f_i(x) = s_i + r_i \cdot x$ 并输出三份秘密份额。以 Student1 ($s_1 = 10$) 为例，其多项式为 $f_1(x) = 10 + r_1x$ ，三份份额分别写入 student_1_1.txt、student_1_2.txt、student_1_3.txt。

步骤 2 中，每个学生汇总收到的 3 份份额，计算聚合值 $d_j = f_1(j) + f_2(j) + f_3(j) \bmod p$ ，保存到 d_j.txt。步骤 3 中，计票员读取 d_2, d_3 ，用拉格朗日插值恢复 $F(0) = s_1 + s_2 + s_3 = 60$ ，再计算：

$$\text{avg} = 60 \times 3^{-1} \bmod p = 60 \times 333333336 \bmod p = 20$$

协议正确执行，最终输出平均值为 **20**，与预期 $(10 + 20 + 30)/3 = 20$ 完全一致。

正确性验证：计票员持有 d_2, d_3 ，通过拉格朗日插值恢复 $F(0) = 60$ ，再计算 $60 \times 3^{-1} \bmod p = 20$ 。全程 Student1 的数据 10 只以份额形式出现，计票员无法从 d_2, d_3 中推断出任何一个人的原始数据。

6 安全性分析

6.1 秘密性 (Privacy)

每个学生只知道其他学生发给自己的一份份额，不知道对方多项式的随机系数 r_i ，因此无法还原他人的秘密。从信息论角度，单一份额 $f_i(j)$ 均匀分布在 $\mathbb{Z}_p$ 上，与秘密 s_i 在统计上独立，任意少于 2 份份额不提供关于 s_i 的任何信息。

6.2 正确性 (Correctness)

多项式加法同态性保证：

$$F(x) = f_1(x) + f_2(x) + f_3(x), \quad F(0) = s_1 + s_2 + s_3$$

每个学生的聚合值 $d_j = F(j)$ 是总和多项式在点 j 处的值。由拉格朗日插值定理，任意 2 个点足以唯一确定一次多项式 $F(x)$ ，从而精确恢复 $F(0)$ 。

6.3 隐私性 (Zero-knowledge for the counter)

计票员拿到的是两个聚合值 d_2, d_3 ，即总和多项式 $F(x)$ 在两点处的值。通过拉格朗日插值，计票员能且仅能恢复出 $F(0) = s_1 + s_2 + s_3$ （以及平均值），无法从两个方程中分离出三个未知数 s_1, s_2, s_3 ，因此任何单个学生的数据都是受保护的。

安全性总结：

- 任何少于 2 个学生的聚合值都无法重构总和（(2,3) 门限保证）
- 计票员重构的是三个数据的总和，无法推断单个数据
- 每个学生只接触自己的原始数据和来自他人的份额，不接触他人原始数据
- 随机多项式系数 r_i 保证了份额的不可预测性

7 实验中遇到的问题

7.1 模数选取对结果的影响

一开始使用较小的模数 p 测试时，发现平均值计算结果与预期不符。原因是在有限域 $\mathbb{Z}_p$ 中，“除以 3”等价于“乘以 3 的乘法逆元 $3^{-1} \bmod p$ ”，而不是整数除法。如果数据之和不能被 3 整除（整数意义下），结果会是一个模意义下的“等价值”，只有在 p 足够大时才能确保结果落在合理范围内。

解决方案：选用大素数 $p = 1000000007$ ，在实验的数据范围内保证结果的直观正确性。同时明确约定参与方输入的数据之和应能被 3 整除，或在协议说明中指出结果是模意义下的平均值。

7.2 拉格朗日插值中的负数取模

Python 的取模运算对负数的处理与数学上略有差异：Python 中 $(-1) \bmod p = p - 1$ （结果非负），与数学上的模运算一致，但在中间计算步骤中出现负数时需要小心。

具体地，拉格朗日基函数中有 $-x_j/(x_i - x_j)$ 项，当 $x_i - x_j < 0$ 时，先取模再求逆会引入符号错误。最终的解决方法是统一对每次乘法结果取模，依赖 Python 的非负取模行为来正确处理符号。

8 总结

本次实验基于 Shamir 秘密共享实现了安全多方平均值计算协议，直接参照教材实验 4.3.3（投票统计）的协议结构，将统计投票总票数推广为计算任意私有数据的平均值。

协议的核心在于多项式加法的同态性：每一方先用随机多项式对自己的秘密进行秘密共享，各方在收到份额后本地相加，得到的聚合值恰好是总和多项式在对应点处的值，最终由计票员用拉格朗日插值恢复总和并除以参与方数量，完成平均值计算。

通过本次实验，深入理解了 (t, n) 门限秘密共享的构造原理、拉格朗日插值在有限域上的应用，以及如何利用多项式加法同态性设计安全多方计算协议，体会到秘密共享作为安全多方计算核心工具的重要价值。

附录：运行说明

环境要求

- Windows 11 操作系统
- Python 3.9 或更高版本
- 无需额外依赖库

运行步骤（自动化）

1. 激活 conda 环境: `conda activate new`
2. 进入目录并运行测试: `python run_test.py`

运行步骤（手动）

1. 三个学生分别运行 `ss_student.py`, 依次输入 id (1/2/3) 和私有数据
2. 三个学生分别运行 `compute_local.py`, 依次输入 id (1/2/3), 进行本地聚合
3. 计票员运行 `average_counter.py`, 输出平均值

文件清单

- `ss_function.py` 一通用函数（快速幂、多项式构造与求值、拉格朗日插值）
- `ss_student.py` 一学生端秘密份额分发
- `compute_local.py` 一学生端本地聚合
- `average_counter.py` 一计票端平均值计算
- `run_test.py` 一自动化测试脚本